

Hermes: Efficient Serving of LLM Applications with Probabilistic Demand Modeling

Abstract

Applications based on Large Language Models (LLMs) have become increasingly popular in recent years. To accomplish a real-world task, LLM applications often issue a series of correlated LLM and non-LLM requests, with dynamic demands on diverse backends. Facing such characteristics, existing systems are inefficient when serving LLM applications due to long queuing delay and low backend quality. In this paper, we notice that the resource demands of an LLM application can be modeled in a *probabilistic* and also *structured* manner, i.e., with a *probabilistic demand graph* (PDGraph). We then propose Hermes, which leverages the PDGraph model for efficient serving of LLM applications. Hermes applies the Gittins policy to determine the scheduling order that minimizes the average application time despite probabilistic demands. It also uses the PDGraph model to help select better backends as well as to prewarm the cold backends. Experiments with diverse LLM applications confirm that Hermes can effectively improve the application serving efficiency, reducing the average completion time by up to 80%.

1 Introduction

Large Language Models (LLMs) [16, 25, 72] have been widely adopted in various domains [19, 24, 68]. Yet, a single LLM request is often deficient for real-world tasks. For example, AI agents may repeatedly submit an LLM request multiple times to mitigate hallucinations [40], and LLM requests may be augmented with external requests (e.g., knowledge retrieval [46], code testing [3], tool calling [11, 64]) to possess the capability to handle diverse tasks. We call such a set of correlated LLM and non-LLM requests—which collaborate to accomplish a user task—as an LLM application. LLM applications are prevalently hosted as a cloud service [2], and it becomes critical to serve them efficiently in clusters.

Compared with traditional workloads, LLM applications have two distinct characteristics. First, the resource demand of LLM applications—like the text generation length and the total inference number—are dependent on the runtime inputs and uncertain a priori. Second, serving LLM applications often requires coordinated provisioning of diverse resource backends (to support both LLM and non-LLM computations) [3, 46]. Third, LLM applications may have heterogeneous service objectives—some focus on end-to-end mission completion time [4, 42], yet some others care about instantaneous token generation speed [50]. □

When serving LLM applications, existing works fail to work well in both inter-application scheduling and intra-application execution. Most works in the literature focus on accelerating individual LLM inferences without awareness of the inter-request correlations [17, 22, 26, 44, 51], suffering suboptimal end-to-end performance due to inter-request delay. While some recent works can improve the performance of LLM applications [29, 49, 50, 70], they focus on specific application types (e.g., chatbot) or optimization aspects (e.g., KV cache reuse), without dealing with the common characteristics of LLM applications. Consequently, due to demand dynamicity, advanced scheduling algorithms like Shortest-Remaining-Processing-Time (SRPT) cannot be adopted when scheduling multiple LLM applications, nor can it be feasible to support diverse objectives. Meanwhile, without coordinated resource provisioning, an LLM application may be impeded on any resource backend due to contention or improper configuration.

In this paper, we seek to enhance the scheduling and execution efficiency of LLM applications, and the key is to leverage request correlations. To be specific, we propose an abstract called *coinference*, which organizes the correlated inferences of an LLM application in proper structure, and also records the non-LLM resource demands between LLM inferences. We note that with the built-in [2] or customized [5] notations in LLM requests, it is easy for the LLM backend to identify the correlated inferences belonging to the same application. The coinference abstraction can help to directly optimize the application-level scheduling objectives. Moreover, for a given LLM application, our empirical study shows that its resource demand is relatively stable across different trial runs with diverse inputs; therefore, with coinference-level demand modeling, we can more precisely predict the demand information of the entire application as well as of each request, which can further facilitate application scheduling and resource provisioning.

Based on the coinference abstraction, we design *Hermes*, a coinference-centric serving system for LLM applications, with the objective of achieving improved efficiency in both inter-application scheduling and intra-application execution. In our Hermes design, we address the challenging characteristics of LLM applications (i.e., demand dynamicity, objective diversity, and resource hybridity) with three modules: coinference demand modeling, inter-coinference scheduling, and intra-coinference resource provisioning.

First, we devise a general and precise method for coinference demand modeling. For generality, we model a coinference as a list of *stages*, where each stage represents a func-

tional unit with one or multiple parallel requests. In each stage, we record a series of properties to express request dependencies, looping status and even non-LLM demands. For preciseness, we choose to record the resource demands in each aspect (e.g., input/output token length, request parallelism) as a distribution rather than a single constant, trying to preserve the panoramic demand information; we also notice the demand correlations between dependent requests and further adopt correlation analysis to enhance the demand prediction accuracy at runtime.

Second, we design a scheduling algorithm that can minimize the average completion time of LLM applications, and further extend it to support diverse objectives. Confronting uncertain demands with distribution-like description, the classical SRPT algorithm is no longer optimal. For scheduling LLM applications, Hermes instead applies Gittins policy [10, 60], which has been proven optimal for jobs with unknown demand but known demand distributions. Moreover, when concurrently serving applications with SLO objectives (e.g., on deadline or instantaneous generation speed), Hermes keeps identifying and prioritizing the ones with a high risk of SLO violation, and the remaining applications are scheduled under the default SRPT-like policy.

Third, for the efficient execution of individual applications, we propose coinference-aware clairvoyant resource provisioning, applying the demand information from coinference modeling for management of the diverse resource backends. When managing the storage space of KV cache [45] and LoRA adaptors [36], we propose clairvoyant cache management to determine the cache eviction order based on the coinference priority and the prefetching moments based on the profiled distribution of inter-stage gaps. When managing non-LLM backends like CPU (to launch docker container) and network (to conduct RAG), we propose clairvoyant queueing that propagates the coinference priority to each resource backend. In cases with multiple LLM backends with heterogeneous batch size configurations, we propose clairvoyant backend selection, which means adaptively selecting the backend for each stage based on the profiled request parallelism. These methods can ensure that an LLM application can have fast completion without being impeded on any resource backends.

We have implemented Hermes with over 4,100 lines of Python code atop vLLM, a popular LLM serving framework. We evaluate Hermes performance against mainstream schedulers with a diverse set of modern LLM applications. Experimental results show that Hermes can improve the average application completion time by up to 90.1% while providing a near-optimal SLO guarantee. Ablation studies also confirm the effectiveness of our approaches in tackling demand dynamics as well as in efficient resource provisioning on diverse backends.

In summary, this paper makes the following contributions:

- A

- B
- C

2 Background and Motivation

2.1 LLM Applications: A Primer

Increasing popularity of LLM applications. Large language models (LLMs) [16, 25, 72] can potentially revolutionize various domains [24, 32]. Recently, LLM workloads increasingly evolve into the form of a compound workflow [7], and below are the typical reasons:

- *To enlarge the input processing range.* For example, the input context windows of typical LLMs are often of limited sizes (e.g., less than 10M tokens) [20, 48], thus processing a large document would require issuing multiple parallel inference requests.
- *To improve the output quality.* LLMs are plagued by the problem of *hallucination* [40], meaning that the output of a single LLM request may be unreliable, and some additional LLM requests are then required to ensure LLM output quality (for example, with FacTool [21] verifications or with self-reflection [41]). This also aligns with the recent trend of *inference time scaling* [27].
- *To integrate diverse capabilities.* The built-in knowledge and interaction modality of LLMs are often insufficient in diverse realistic occasions, thus various non-LLM components (e.g., knowledge retrieval [46], third-party tools [64] or other small models [63]) are commonly adopted to augment LLM capabilities [11], as integrated in OpenAI Assistants [2] and multi-agent frameworks like MetaGPT [34].

To summarize, for a real-world task, to handle large input size, ensure output reliability and support diverse capabilities, LLM practitioners often conduct multiple inferences together with the necessary non-LLM operations, which we call an *LLM application*. LLM applications are rapidly emerging in recent years [32, 49, 70], and would become a mainstream workload type in the GenAI era.

Cloud-based serving of LLM applications. LLM applications, just like OpenAI assistant API [2], are usually hosted on the clouds for ease of user access. The code files of those applications are maintained by the service provider, which would be launched when users submit their task inputs. When executing an LLM application, the LLM requests and non-LLM operations need to be dispatched to proper resource backends (e.g., GPU servers or—for code testing in sandbox environment—docker servers). A production service provider concurrently serves many LLM applications from different users [38, 66]; for good user experience and high service profit, it is crucial to attain efficient serving of LLM applications. To

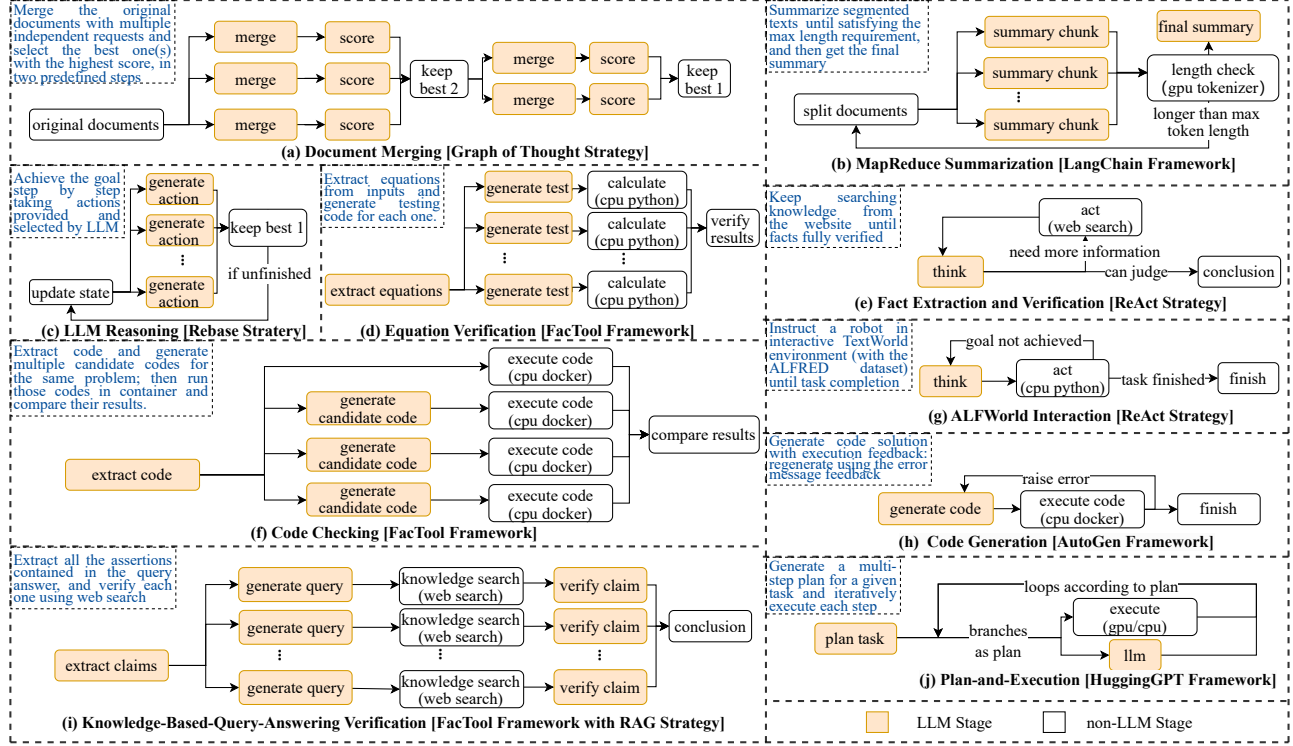


Figure 1: The functionalities and structures of ten typical LLM applications. For conciseness, hereafter we use the uppercase letters in each application name as the abbreviation representing that application.

be specific, by launching an LLM application, users primarily care about the application completion time (e.g., when the high-quality output is finally returned back).

Distinct characteristics of LLM applications. LLM applications differ substantially with those traditional workloads in OS or big data domain—in that the resource demands of an LLM application are highly dynamic, and that the resource backends required to serve an LLM application are also quite diverse. To elaborate, we create a comprehensive workload suite of LLM applications as shown in Fig. 1, which involves ten representative applications built with specific frameworks (e.g., FacTool [21] and AutoGen [4]) or following certain control strategies (e.g., Graph of Thought [15] and ReAct [41]). With Fig. 1, we next elaborate the two distinct characteristics of LLM applications in greater detail.

1) Demand Dynamicity. The resource demand of LLM applications exhibits strong dynamicity—in both demand structure and quantity. First, the application *structure* (i.e., the number and organization of the constituting requests) may be uncertain in advance. Although some applications (like the hard-coded inference workflows with LangChain [71]) have a fixed structure, for some other applications with ReActing [41, 75] or LLM-planning [63], the application structure can only be determined at runtime based on the inference contents. Second, for each individual request, as pointed out by many existing works [29, 74], both the input and output token

lengths are dynamic. This implies that the request durations (which are positively correlated with the number of tokens) are also difficult to predict a priori.

2) Backend Diversity. Serving an LLM application often involves a diverse set of resource backends—in both LLM backends and non-LLM backends. Different LLM requests, with heterogeneous cost-preferences or professional requirements, may desire different LLM models [18] or fine-tuned LoRA adapters [65]. Regarding the non-LLM operations, a diverse set of backends may also be needed. For example, in the code-generation application, the LLM-generated codes need to be tested on CPU backends (e.g., with a docker container [3] to provide an isolated environment). Meanwhile, knowledge retrieval [46] requires the participation of search engines. Besides, arithmetic (e.g., a calculator or a symbolic solver as adopted in AlphaGeometry [1]) and multi-modal (e.g., Stable Diffusion model [58]) tools may also be built-in components of LLM applications [11]. Failure to provide any of such required backends would result in postponed completion of the entire LLM applications.

Next, we will show that the above characteristics render existing LLM serving systems inefficient for LLM applications.

2.2 Limitations of Existing Serving Practices

For LLM applications, existing practices fail to provide efficient serving performance. A mainstream framework goes to

vLLM [8], which serves incoming LLM requests in a FCFS manner—without awareness to LLM applications. Consequently, the constituting requests of an LLM application may be executed in an interleaved manner, compromising the application completion time. Recently, some works do notice the need to optimize end-to-end performance of LLM applications. Parrot [49] employs a simple heuristic that schedules the requests of an application together; Teola [70] conducts multiple compilation optimizations to speed up the execution of a single LLM application. However, these works do not focus on inter-application scheduling (still FCFS), and the applications they target are also limited in representativeness—without considering demand dynamicity (both assume a static application structure programmed with dedicated APIs) or backend dynamicity (Parrot does not consider non-LLM backends and Teola only considers retrieval modules).

In summary, no systems have been designed to efficiently serve general LLM applications involving dynamic demands and diverse backends. Specifically, our empirical study shows that there exist a series of performance bottlenecks when serving LLM applications with existing strategies.

Inefficiency due to long queueing delay. In our testbed measurement, we serve multiple LLM applications with a LLaMA-7B model hosted on two V100 GPUs. The first type of performance bottleneck goes to the queueing delay on both LLM and non-LLM backends.

1) *Queueing delay on LLM backends.* We first let a DM application compete with a KBQAV application in the waiting queue, where DM comes earlier. As shown in Fig. 2a, under the default FCFS policy, DM is scheduled first and the average application completion time (ACT) is 52.8s; however, since DM is a larger application than KBQAV, an ideal scheduler shall execute KBQAV first—yielding a smaller avg. ACT of 40.3s. Note that, LLM backends involve more than just computing resources—the cache space (used for storing the KV-cache [45] or LoRA adaptors [36]) may also be contended, and similar scheduling problem also exists.

2) *Queueing delay on non-LLM backends.* Enforcing the ideal priority only on LLM backends is not enough. Among the diverse non-LLM backends, we for example focus on the docker backend, which is used by both CG and CC applications. We let a CG application comes after the CC application and compete for both LLM and docker resources. Since CG is smaller, it has a higher priority on the LLM backends; yet on the docker backend—managed by an independent Kubernetes scheduler—they still follow the FCFS policy. As shown in Fig. 2b, in that case, the docker launching request of the CG application has to wait behind CC, suffering a completion delay of 46.4%.

Inefficiency due to low backend quality. Even if an LLM application is granted the highest priority without queueing delay, its execution may still be inefficient due to improper backend selection or backend warm-up delay.

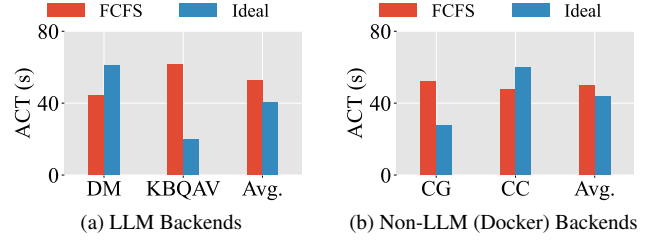


Figure 2: LLM applications may be delayed due to improper queuing order in both LLM and non-LLM backends.

1) *Improper backend selection.* A service provider may provision multiple entities with different configurations for a single backend type to avoid interferences [35]. A typical example is that the LLM API dispatches the incoming LLM requests to different LLM servers each with a distinct batch size [49]. In general, LLM servers with a small batch size can make fast request response, and with large batch size high throughput; blindly scheduling an incoming request to an arbitrary backend may lead to inefficient execution at the application level. As shown in Fig. 3a, we build two applications, one with a single request and the other with 10 parallel requests (all with an input/output length of 1000/100). In Fig. 3b, compared with the ideal case, dispatching the two applications to improper backends lead to a delay of $1.34\times$ and $3.16\times$, respectively.

2) *Warm-up delay of cold backends.* As elaborated in Sec. 2.1, LLM applications may potentially demand a wide array of diverse backends (even user-defined ones like OpenAI function calling [9]), and it is inefficient to make such backends standby all the time. Therefore, it is possible that the backend to use be cold at request time—meaning that additional time needs to be taken for backend preparation. For example, to prepare the LLM backends, we need to load the LLM model into the GPU HBM, and in some cases with token reusing or model customization, the KV-cache and LoRA adaptors also need to be loaded. For non-LLM backends, it takes time to warm up the desired docker container, and the DNN models for multi-modal recognition or generation also need to be loaded into a GPU. In Fig. 4a and Fig. 4b, we show the typical warm-up durations of a series of backend types, normalized by the inference time of a standard LLM request (with an input/output length of 1000/100). It shows that such warm-up overheads are indeed substantial (up to $18\times$).

Lessons. From the above empirical study we can learn that, for efficient serving of LLM applications, we need to apply a proper queuing order on each backend, and meanwhile dispatch each request to proper backend warmed up a priori. All such optimizations require the demand knowledge of LLM applications.

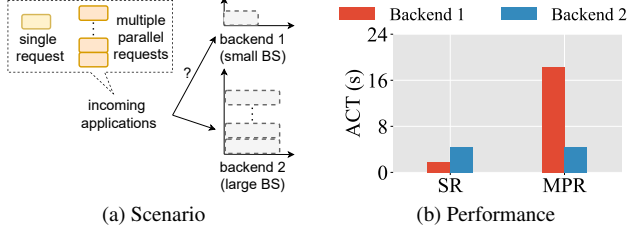


Figure 3: Suppose there are a single-request application (SR) and an application with multiple parallel requests (MPR), and two backends respectively configured with small and large batch size. Compared to the ideal dispatching strategy (SR to small-BS backend and MPR to large-BS backend), improper dispatching suffers substantial slowdown.

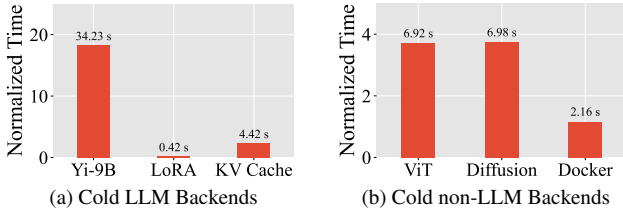


Figure 4: Costs to warm up code backends (normalized to a typical LLM inference time). The LoRA is for LLaMA-7B with rank 8 and the KV cache size is 128K (loaded to V100 GPUs). The docker image is python:3.10-slim.

3 Study on Demand Correlation of LLM Apps

Although the demands of an LLM application in non-deterministic, this however does not mean that they are totally agnostic and LLM applications must be served in the dark. On the one hand, improving the serving quality does not require the exact demand information [28] (a probabilistic description may suffice); on the other hand, LLM applications maintained by a service provider are usually recurring workloads frequently launched by public users, making it possible to make thorough performance profiling a priori.

As mentioned in Sec. 2, an LLM application is written as a fixed code segment hosted on the cloud; such a code segment describes the control flow of different *functional units* composing that LLM application: some units issue requests (with the same system prompt) to the LLM backends, yet some other units launch non-LLM operations. Based on such structured view of functional units, it is possible to leverage the unit correlations to estimate the demands of each LLM application, as accurate as possible. To be specific, we find that such demand correlation exists in two aspects. First, for each functional unit, its resource demands across different trial runs are often correlated. Second, in the ongoing run of an LLM application, the resource demands of dependent functional units are also correlated. Next we elaborate them

with empirical measurements. [mention prediction works?]

Demand correlation across different runs (inputs) for a specific functional unit. We run five LLM applications from Fig. 1 each for 100 times (with different inputs sampled from the original dataset). Fig. 5 records the execution statistics on input/output token lengths and request parallelism of an arbitrarily selected functional unit of each application. As shown in Fig. 5, although the resource demands are non-deterministic, they do exhibit salient stability across different trial runs. For example, the output token length of the `merge` request in Document Merging application is around 1000, yet for the `generate-query` request in KBQAV application it is between 10 and 50. That phenomenon is indeed reasonable because demand volume is to some extent an inner property based on the usage scenario of an LLM application: the output of the `merge` request is a document, yet for the `generate-query` request it is a short query—thus its length distribution has a much smaller mean value and also a much smaller deviation. Such stability also exists in the request parallelism aspect. This way, the historical statistics can help to narrow down the estimation range and facilitate accurate prediction.

Demand correlation between dependent functional units in the ongoing run. Apart from such cross-round correlations, due to the structural dependencies among functional units, the resource demands of a unit may be correlated to the upstream ones. We note that there typically exist three cross-unit demand correlation patterns:

1) A unit’s request input length may be correlated to the upstream unit’s request input length and output length. For example, for the Document Merging application, the input of each request in the `scoring` unit is a superset of a request’s output in the (upstream) `aggregate` unit (plus a fixed system prompt). Meanwhile, for the looping unit, downstream requests would share the same prompt template as the upstream ones, indicating similarity in input length.

2) A unit’s request output length may be correlated to its input length as well as the upstream unit’s request output length. For example, for the `generate-code` unit in Code Checking application, a more complex input task (often with a longer prompt) usually yields a longer code segment; so for other LLM-tasks like translation and article correction/polishing. Meanwhile, for the looping states, the request output lengths are also similar across units.

3) A unit’s request parallelism may be correlated to the upstream unit’s parallelism. For example, as shown in KBQAV applications, for each inference request in the (upstream) `generate-queries` unit, a corresponding inference request would be launched at the (downstream) `verify-claim` unit.

To verify the existence of above correlation types, we resort to *Pearson Correlation Analysis* [61] based on our profiled data over 100 trial runs. We divide the demand range to 10 buckets, and use $P(X = i)(i = 0, 1, \dots, 9)$ to represent the probability that the demand quantity variable X

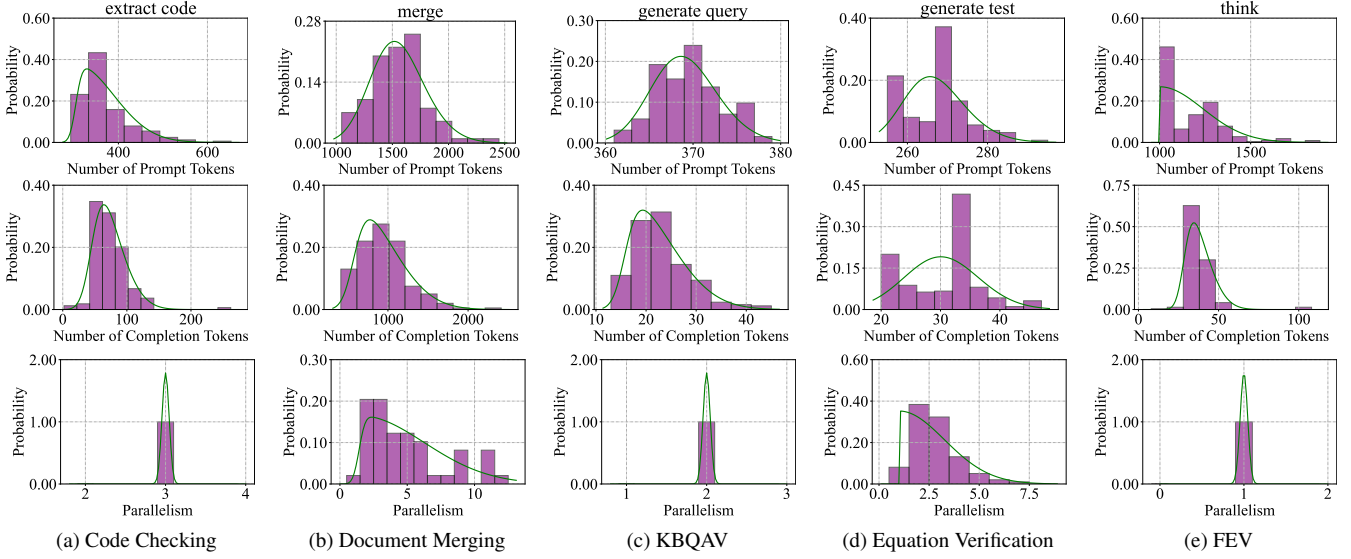


Figure 5: Prompt and completion token length distribution for an arbitrarily-selected request (marked at the top) in five applications each over 100 trial runs. In each case, we divide the length range into 10 buckets, and calculate the value appearance probability in each bucket (accompanied by the fitted curves assuming skewed Gaussian distribution for reference).

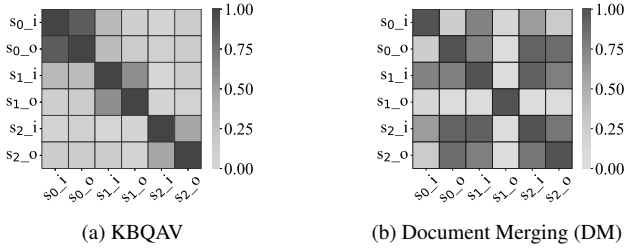


Figure 6: The Pearson correlation coefficients between demands of dependent functional units. We show three units respectively in KBQAV and Document Merging, where s_{x-i} , s_{x-o} represent the input/output token length of the x -th unit.

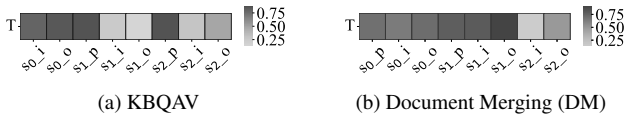


Figure 7: The correlation coefficients between an application's overall duration (T) and each functional unit's demands (input/output token lengths and request parallelism; s_{x-p} represents the parallelism of the x -th unit).

falls into bucket- i . Then we calculate the Pearson correlation coefficient between two demand variables X and Y as $\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\mathbb{E}((X-\mu_X)(Y-\mu_Y))}{\sigma_X \sigma_Y}$. In Fig. 6, for KBQAV and Document Merging, we further depict the Pearson correlation coefficients between any two demand variables (each represent the input/output length of a stage). For KBQAV, each stage's output length is strongly correlated to its input length; yet for Document Merging, each stage's input length is highly-related to the upstream stage's input length.

The above correlation analysis facilitates unit-level demand

estimation and helps to realize high-quality backend provisioning; yet, for inter-application scheduling, we also need to efficiently estimate the overall demand of an LLM application. Fig. 6 already indicates that the local demands of a functional unit may affect the downstream ones—and thus may further affect the overall demand of entire application. In Fig. 7, we measure the Pearson coefficients between the local demand factors and the overall duration (with the durations of all the LLM requests and non-LLM operations summed up). Fig. 7 confirms that certain local demand values (i.e., *critical local demands*) do correlate with the global demand. Therefore, it is possible to refine the demand estimation of the entire application based on the runtime information of those critical local demands.

To summarize, although the demands of an LLM application are not deterministic, we can use a probabilistic distribution to estimate those demands, and various correlation relationships can additionally be leveraged to refine the probabilistic estimation to a more accurate range.

4 Hermes Design

We present Hermes, an efficient serving system for LLM applications hosted on clouds. As shown in Fig. 8, Hermes maintains a knowledge base storing the profiled execution information for each application. When serving an LLM application, Hermes creates an app-handler to monitor its execution process; that app-handler requests the profiled demand model from the knowledge base, and submits each constituting requests—with annotated hints on scheduling and backend-provisioning—to the scheduler.

optimal in that regard is *shortest-remaining-processing-time* (SRPT) [59]. However, naively applying SRPT is not appropriate for our problem: SRPT relies on deterministic demand knowledge to make scheduling decisions, yet the resource demands of LLM applications are uncertain a priori (our PD-Graph strategy mitigates but does not totally eliminate such uncertainty). Although it is possible to use the expected value (mean) of a demand distribution to emulate SRPT scheduling, this method often fails to work well due to its blindness to instantaneous execution progress. For example, for a request with an expected output length of 20 tokens, it is possible that its token generation process does not stop even after 100 tokens. In that case, we need to timely refresh the demand estimation based on the latest progress rather than sticking to the prior expectations (otherwise the remaining processing time—the expected execution time minus the executed time—may ironically become negative). Therefore, we need to introduce uncertainty-awareness in designing our scheduling algorithm.

The problem we now face—minimizing the average completion time for jobs with *unknown durations* but *known duration distributions*—has been well studied in the literature, for which the *Gittins* policy [10, 30, 60] has been proven optimal. The basic idea of Gittins policy is to calculate a *Gittins rank* for each job—based on its executed time so far and its size distribution—as the scheduling priority. Specifically, let an application’s duration distribution be $\mathcal{D}$, and it has been executed for a time period of a , then its Gittins rank G can be expressed as:

$$G(\mathcal{D}, a) = \inf_{\Delta > 0} \frac{\mathbb{E}[\min\{X_{\mathcal{D}} - a, \Delta\} \mid X_{\mathcal{D}} > a]}{\mathbb{P}\{X_{\mathcal{D}} - a \leq \Delta \mid X_{\mathcal{D}} > a\}}, \quad (1)$$

where $X_{\mathcal{D}}$ is the random variable under $\mathcal{D}$. Given a service budget Δ (i.e., the number of additional tokens allowed to be generated henceforth), the denominator in Eq. 1 represents the possibility that the generation process can finish before Δ (which contributes to small average completion time), and the numerator represents the corresponded resource consumption (the cost paid to serve it). Note that G is always no larger than the maximum value in $\mathcal{D}$, and it can be viewed as the *true runtime estimator of the remaining processing demand* when determining the scheduling priority: existing works [30, 60] have shown that this method can theoretically yield the minimal average completion time. In Fig. 10, we show a case where the Gittins policy yields a different yet better scheduling order than SPRT (in practice, we update the estimation of Gittins index after each bucket period).

Applying application-level scheduling priority to diverse resource backends. Recall that in Sec. 2.2, we have learned that merely applying the desired priority of an LLM application to its LLM backends is not enough—a high-priority application may still be impeded on a non-LLM backend if that non-LLM backend is monitored by an independent scheduler using the default FCFS policy. Therefore, Hermes adopts a simple yet effective principle that always propagating an

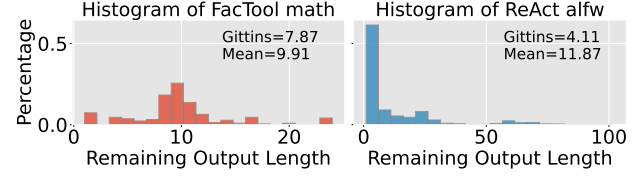


Figure 10: Given two LLM applications, Math_FacTool and ALFW_ReAct, under SPRT it is the former that is scheduled first, yet under the Gittins policy it is the latter (with a lower Gittins rank). Regarding the probability to complete within three additional iterations, for Math_FacTool it is 7%, yet for ALFW_ReAct it is 48%; this suggests that prioritizing ALFW_ReAct indeed has a higher reward in practice.

application’s priority to all its involved backends. This way, it can be ensured that both the LLM and non-LLM components of an application with the highest priority can be serviced without any queuing delay.

Extend to other efficiency³ objectives. Regarding efficient LLM application scheduling, apart from the ACT criterion, users may also associate a deadline with their application request, and expect that the final output can be returned before that deadline; in that cases, the efficiency criterion is the ratio of applications that can satisfy the deadline requirements. For deadline-based scheduling, we note that PDGraph-based demand modeling can also help to attain better performance—surpassing the demand-agnostic scheduling methods exemplified by Earliest-deadline-first (EDF) [14]. On the one hand, EDF always prioritizes the application with the earliest deadline, yet an earlier deadline does not mean a higher urgency. Based on the probabilistic demand modeling, we can calculate the risk that an LLM application—if scheduled instantly—finishes after its specified deadline. Then we can prioritize the application having the highest risk for better overall performance—essentially emulating the effect of *Least Slack Time First* (LSTF) [23]. On the other hand, for an application with a high deadline-violation risk (default to 0.25), we can enable admission control to direct reject it at submission time—this can avoid wasting resources on applications that finally violates the deadline, which is important for production clusters [56].

4.3 Optimizing Backend Quality

So far we focus on determining the scheduling order of different applications, yet as elaborated in Sec. 2.2, we also need to improve the backend quality for each scheduled application—by selecting a proper backend and prewarming cold backends. In this part we show how PDGraph can help in both aspects.

PDGraph-based backend selection. For dispatching a request to proper backend, we propose a simple heuristic based

³[potential of WFQ; demand info also facilitate fairness]

on the PDGraph. Suppose there is a latency-sensitive backend (with batch size $B_{\min}$) and a throughput-sensitive backend (with batch size $B_{\max}$)—emulating a “power-of-two-choices” scheduling space [67]; given a request from a functional unit, we schedule it to the latency-sensitive backend if the mean value of the estimated request parallelism of that functional unit is larger than $B_{\min}$ (so that all the requests of that unit can be executed in a single batch), and otherwise to the throughput-sensitive backend. Our evaluations show that this heuristic can indeed work well in practice.

PDGraph-based backend prewarming. As elaborated in Sec. 2.2, cold backends would delay the completion of an LLM application. With PDGraph, Hermes can estimate the arrival of the downstream requests as well as their desired backend types. By prewarming cold backend to be used by the downstream unit before the completion of the current unit, we can get rid of the warm-up delay from the critical path. However, backend prewarming is not free-of-charge. On the one hand, a functional unit may have multiple downstream units, and it is possible that the prewarmed backends are never used; on the other hand, the current unit may complete much later than expected, meaning that the prewarmed backend would wait idly for a long time. Therefore, as shown in Fig. 11, there exists a trade-off in determining the prewarming triggering moment: more aggressive prewarming can help attain faster application completion—yet at the cost of larger resource wastage.

We then introduce a knob K , called *expected-prewarming-effectiveness*, to tune that trade-off. To be specific, given a running function unit, if one of its downstream backends is not active, we then determine *whether* and *when* to trigger prewarming with an analytical method. We let p_s be the probability to select the target unit, t_c be the actual completion time of the current unit, t_s be the moment to start prewarming, and t_p be the duration of the prewarming operation. Then the probability that prewarming is effective (i.e., when the target downstream request finally arrives, its backend is already well-warmed) can be expressed by $p_e = p_s * P(t_c > t_s + t_p)$. Given the knob K , if $p_s < K$, then Hermes does not trigger prewarming; otherwise, it triggers prewarming at the time such that $p_e = K$. Such a knob K can be deemed as a kind of service level agreement: a more significant application (e.g., from premium users) can be assigned with a smaller K value.

The above prewarming technique applies to the non-LLM backends like Docker and DNN tools, and also applies to the cache resources in LLM backends. In fact, the cache space on LLM servers is significant for efficient LLM serving. Existing works [11, 45] have shown that KV-cache can remarkably improve the token generation speed for requests sharing the same prompt tokens. Meanwhile, Low-Rank Adaptation (LoRA) [36] is prevalently adopted to support LLM serving with customized models; those LoRA adaptors take non-negligible time to load and should thus be also cached in memory [47]. In that sense, preloading the desired KV cache

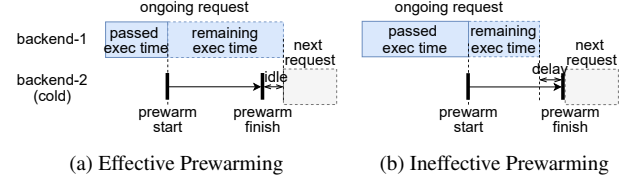


Figure 11: [Replace to a toy example showing the trade-off.]

or LoRA is also necessary for the efficient execution of LLM applications.

5 Implementation

We have implemented Hermes with 4100 lines of Python codes. The profiled application execution information is stored in a large json file[the json file is only 106KB] called KnowledgeBase, and as showed in Fig. 8, the key components we implemented are the AppHandler and the HermesScheduler. Additionally, we build a comprehensive benchmark suite consisting of 11,300 lines of Python code, which includes all of the application types depicted in Fig. 1.

Workflow. With Hermes, users first request the AppHandler to launch an LLM application. That request contains the application name, input as well as some additional information (e.g., deadline or the hyperparameter of expected prewarming effectiveness, with style like the `extra_body` parameter provided by OpenAI [5] and vllm [6] interfaces). Upon receiving such an application-launching request, the AppHandler searches in the KnowledgeBase to get the PDGraph for that application. Then the AppHandler respectively sends each constituting request of that application—with annotated hints—to HermesScheduler. Those hints have two types: scheduling hints (e.g., Gittins index or deadline-violation risk) and backend hints (e.g., the request parallelism or backend specifications). Upon receiving an annotated request, the HermesScheduler calculates a global priority for that request’s application, and routes that request to the local scheduler of the corresponded backend with the priority. Note that the AppHandler also issues backend prewarming requests to the HermesScheduler based on the mechanism described in Sec. 4.3; the HermesScheduler then prepares such backend and registers it as ready-to-use after warm-up completion.

6 Evaluation

6.1 General Setups

Hardware Platform. In preparing the service backends for our workload suite, for LLM backends the models we use are LLaMA-7B (the default model) and LLaMA-13B. The LLaMA-7B model is hosted on two V100-PCIE-16GB

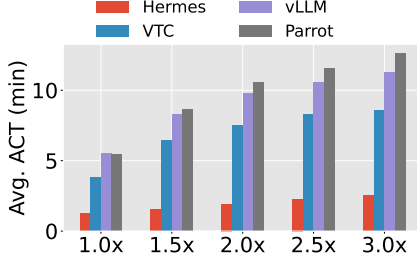


Figure 12: The average ACT under different schedulers with varying application arrival intensities.

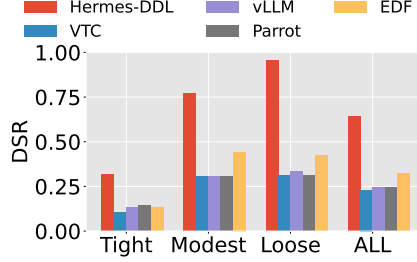


Figure 13: The DDL Satisfactory Ratio under different schedulers.

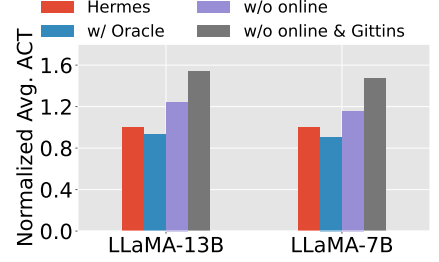


Figure 14: Ablation study on the methods tackling demand dynamics.

GPUs in a tensor-parallel manner, or on a single A100-PCIE-40GB GPU for experiments related to cache management. The LLaMA-13B model is hosted on four V100-PCIE-16GB GPUs. Besides such long-living LLM backends, we prepare the non-LLM backends (i.e., docker backend and DNN backend) in an on-demand manner; the docker backend is hosted on a CPU server with totally 40 cores (each docker container demands 2 cores), and the DNN backend (like the diffusion model) is hosted in another V100 GPU.

Workloads. For our experiments, we create a mixed workload suite with totally 300 LLM applications (each with distinct inputs from the original datasets). Specifically, similar to prior work [39, 55, 79], we set the sampling probability of *small* (EV, FEV, CC, ALFWI and KBQAV—usually less than 1 min), *medium* (CG and PE—usually between 1 and 10 min) and *large* (DM and MRS—usually longer than 10 min) applications to be 72%, 26%, and 2%, respectively. Regarding application submission, given a submission window of T , we let the submission interval follow a Poisson distribution with the expectation be T/N (N is the total application number submitted in experiments).

Baselines. We compare Hermes with three scheduling strategies: vLLM [8], Parrot [49] and VTC [66]. As explained in Sec. 2.2, vLLM adopts the FCFS policy at the request level, and Parrot adopts the FCFS policy at the application level. VTC is a recently proposed scheduler that seeks to fairly serve the LLM requests from different users, which is also agnostic to the application demands. For deadline-based scenarios, we additionally include the Earliest-Deadline-First (EDF) policy [14] as a baseline. We defer the introduction of other baselines to each specific micro-benchmark experiment. Regarding the default hyperparameter setup in Hermes, the threshold on Pearson correlation coefficient (Sec. 4.1) is set to 0.5, the deadline-violation risk (Sec. 4.2) is set to 0.25, the expected prewarming effectiveness (Sec. 4.3) is set to 0.5, and the bucket size (Sec. 4.1 and Sec. 4.3) is set to 10.

6.2 End-to-end Scheduling Performance

Minimizing Average ACT. We first evaluate how Hermes can improve the application serving efficiency measured by the average application completion time. We submit all the 300 LLM applications with the submission window respectively set to 10, 12, 15, 20 and 30 minutes (i.e., the workload intensity respectively be $3\times$, $2.5\times$, $2\times$, $1.5\times$ and $1\times$). As shown in Fig. 12, Hermes can perform much better than vLLM, Parrot and VTC in each case. For example, when workload intensity is $1\times$, the ACT performance of Hermes is 77.0% (76.7% and 66.7%) better than vLLM (Parrot and VTC); when the workload intensity is $3\times$, the ACT performance of Hermes is 77.3% (79.9% and 70.3%) better.

Maximizing DDL satisfactory ratio. We next evaluate Hermes-DDL, a variant of Hermes specifically designed for deadline-constrained scenarios (based on methods in Sec. 4.2). Following the previous experimental setup, we submit the same 300 applications within a 15-minute submission window. Additionally, we assign distinct deadlines to each task by scaling the original execution time with random factors of $1.2\times$ (*tight*), $1.5\times$ (*modest*), and $2\times$ (*loose*), consistent with methodologies employed in prior studies [31, 52]. Fig. 13 reports the Deadline Satisfaction Ratio (DSR, meaning the ratio of applications that can complete before the specified deadline)—for all the applications as well as for applications in each DDL-scaling category. It shows that Hermes-DDL achieves the highest DSR among all the schemes evaluated, delivering a $1\times$ improvement over EDF. This improvement primarily stems from Hermes-DDL’s ability to leverage application demand information, with which it prioritizes the most urgent applications while deferring less critical ones. Meanwhile, it can also proactively reject applications that have a high deadline-violation risk (of over 0.25), avoiding the potential resource wastage.

6.3 Effect on Tackling Demand Dynamics

Setup. Recall that to address demand dynamics, we have adopted multiple techniques including offline profiling, on-

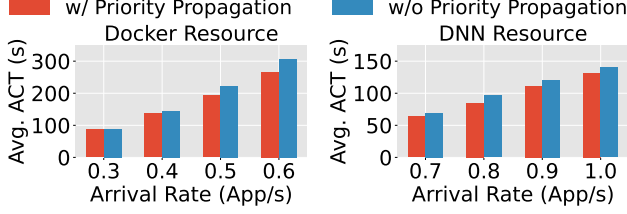


Figure 15: Hermes performance on two different non-LLM backends in cases with resource contention.

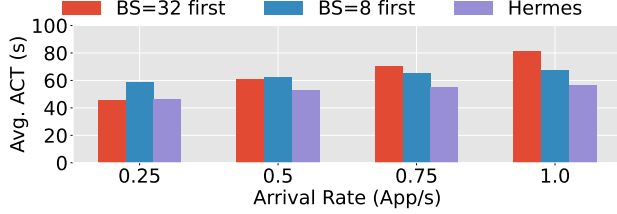


Figure 16: Effectiveness of backend selection.

line refinement as well as the Gittins policy. We now verify their respective effectiveness by comparing the performance between four schemes: Hermes, Hermes without online refinement, Hermes without online refinement or Gittins, and Hermes with Oracle (representing the ideal case where the exact demands are available—by conducting a trial run with temperature set to 0). We conduct this experiment respectively with LLaMA-7B and LLaMA-13B as the LLM backend, and submit 300 applications within a time window of 10 minutes.

Results. Fig. 14 shows the average ACT in each case (normalized by that under Hermes). Without online correlation analysis, the average ACT would be inflated by 15.3% with the LLaMA-7B model. When the Gittins policy is further disabled, that performance degradation would increase to 47.5%. Such results confirm the indispensability of both the correlation analysis and the Gittins policy in tackling demand uncertainty. Meanwhile, the performance gap between Hermes and Hermes-Oracle is less than 10%, suggesting that Hermes can deliver near-optimal scheduling performance for LLM applications. We also notice that in each case, the performance of LLaMA-7B and LLaMA-13B are quite similar.

6.4 Queuing Optim. on Non-LLM Backends

To assess the effectiveness of applying the global order on non-LLM backends (§4.2), we select two LLM applications, CG and PE, which respectively demands docker (to test code in a virtual environment) and GPU (to run the small DNN models) resources. Note that the maximum number of docker containers in our provisioned CPU server is 20, and the maximum number of DNN models concurrently served with the V100 GPU is 15. In each case, we submit the selected appli-

cations under different arrival rates, and Fig. 15 depicts the average ACT. Compared with the default case where each backend queues the request independently in FCFS order, Hermes can reduce the average ACT by up to 13%.

6.5 Effect on Backend Quality Optimization

Effect of PDGraph-based backend selection. We further evaluate the effectiveness of our automatic backend selection method. To that end, we deploy two LLaMA-7B backends each with two V100 GPUs: the batch size for one backend is configured to 32 for high throughput, and the other to 8 for low latency. The schemes compared are Hermes (i.e., try using throughput-first backend for stages with parallelism larger than 8, and try using the latency-first backend otherwise) and two naïve strategy that always preferentially choose a fixed backend (either 8 or 32). We submit 500 LLM applications and study the average ACT of those applications with non-trivial stage parallelisms (i.e., larger than 1). Such experiments are repeated under different application arrival rates (with identical submission order in each case). As shown in Fig. 16, in any case, Hermes can always make the best performance.

Effect of LLM backend prewarming. As explained in Sec. 4.3, prewarming the KV and LoRA cache can facilitate LLM serving. To confirm that, we submit 500 applications to the A100 server within 15 minutes. In particular, we introduce two cache management baselines: LRU and Evict/Prefetch-Waiting-Queue (EPWQ) [29, 65]. LRU does not conduct prewarming, and EPWQ prewarms the KV cache only when the request is already in the waiting queue—which is often too late. Under each method, we measure the overall cache hit ratio (i.e., KV cache be well warmed when the request comes) against different cache sizes (8GB, 16GB and 32GB). As shown in Fig. 17(a), Hermes consistently achieves the highest cache efficiency in each case, improving the overall cache hit ratio by up to $1.11 \times (0.33 \times)$ compared to LRU (EPWQ).

We then similarly evaluate the benefit to prewarm the LoRA cache. We set `max-loras` (the maximum number of parallel processes per iteration) to 10, `max-cpu-loras` (the number of LoRA caches that can be stored in CPU memory) to 20, and use totally 200 LoRA adapters for LLaMA-7B with a rank of 8. We submit a 25-minute workload containing 1,000 applications, with each application randomly assigned to one of the LoRA adapters (all requests of an application use the same LoRA). Fig. 17(b) shows the cache hit ratio also respectively under LRU, EPWQ and Hermes. Compared to the second best (EPWQ), Hermes can increase the cache hit ratio by 21.1%.

Effect of Non-LLM backend prewarming. To evaluate our prewarming strategy in non-LLM backends, we selected two applications, CG and PE, which respectively demands Docker and DNN backends. We separately run each application with varying levels of prewarming aggressiveness (i.e.,

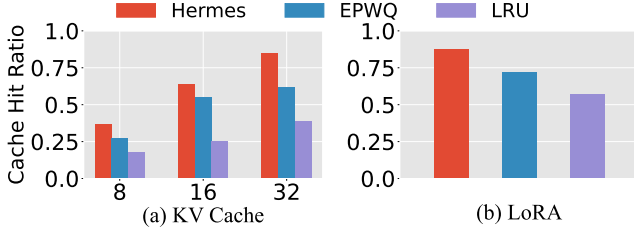


Figure 17: The cache hit ratio of (a) the KV Cache and (b) the LoRA adapter across different cache management strategies.

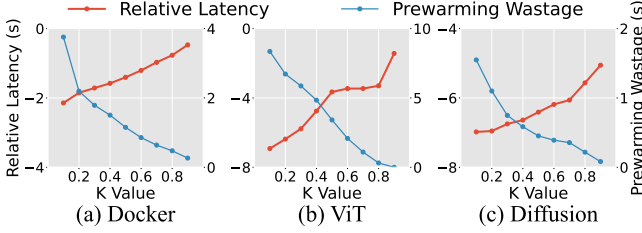


Figure 18: Benefit and cost with different prewarming aggressiveness levels (K) for non-LLM backends.

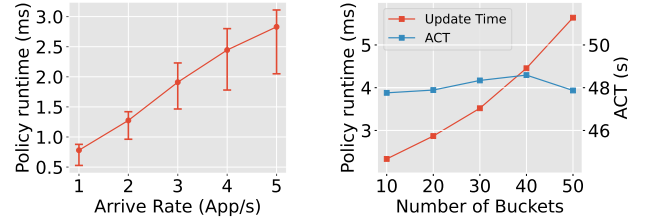
the expected-prewarming-effectiveness hyperparameter, K). Fig. 18 illustrates the average latency reduction and resource wastage due to prewarming, which verifies the effectiveness of prewarming in non-LLM backends (for DNN backends, we only select the PE applications calling a ViT or Diffusion model). Moreover, a smaller value of K can in general yield faster completion, yet at the cost of higher resource wastage—consistent with our analysis in Sec. 4.3.

6.6 Overhead Analysis

The main overhead of Hermes lies in determining the application scheduling priority with Gittins, which is iteratively executed once after a time length of the bucket size (buckets are used in describing the demand distribution as in Fig. 5). In Fig. 19a, we measure the average policy runtime of Gittins under different arrival rates (indicating different scheduling scales—note that 1 app/s is already quite intense surpassing our previous setups). It shows that the scheduling overhead is indeed quite small in each case (less than 3 ms). Further in Fig. 19b, we measure how the Gittins policy runtime varies with the bucket number (duration range divided by bucket size). It shows that using more buckets to describe the demand distribution does not help improve the scheduling performance, but almost linearly increases the policy runtime.

7 Related Work

Accelerating individual LLM requests. Many works seek to accelerate individual LLM requests in a series of aspects. In



(a) The policy runtime of Gittins over different application arrive rates. (b) How Gittins policy runtime and ACT vary with the bucket numbers.

Figure 19: Overhead Analysis.

operator implementation aspect, the FlashAttention [22] and FlashDecoding [33] style algorithms are commonly adopted to realize efficient LLM inference by integrating IO awareness. In *model deployment* aspect, model-quantization [26, 44], paged-attention [45] and prefill-decode-separation [12, 81] methods have been proposed to maximize backend utilization. In *inference algorithm* aspect, continuous batching [76] and speculative decoding methods [17, 51] are also applied to improve the inference throughput by accelerating the decoding process. These works are orthogonal to us.

Scheduling multiple LLM requests. Some recent works do notice the need to optimize scheduling performance at the request level. FastServe [74] adopts multi-level feedback queue for LLM inference serving (yet at the cost of relatively high preemption overhead); Llumnix [69] and LoongServe [73] enable runtime request migration to improve load-balancing and isolation. In the meantime, some other works seek to adopt SJF-like scheduling algorithms [35, 53, 62]. While those methods can reduce the average completion time of requests, they are application-agnostic and, as explained in Sec. 2.2, fail to yield fast application completion. [infercept]

Job demand prediction. Demand profiling is important for efficient job scheduling. In traditional fields, a series of research works have been proposed to make accurate performance prediction with testbed profiling [13, 78] and mathematical modeling [37, 54]. Yet LLM workloads exhibit distinct demand uncertainty due to its generative manner, which is never captured before; some recent methods have been proposed to predict the output token length of an LLM request—with LLMs themselves [43, 57, 62, 80], or with retrieval-based methods [53, 77]. Yet, none of those methods make demand prediction from the application point of view; blind to the request dependencies, they cannot support efficient application scheduling or speculative preparation of diverse backends.

8 Conclusion

[In this paper, we propose Hermes, an efficient serving system designed for LLM applications. Hermes employs a probabilis-

tic demand graph (PDGraph) to model the resource demands of LLM applications. Using the PDGraph, Hermes adopts the Gittins policy to optimize the scheduling performance of LLM applications with probabilistic demands and propagates priorities across multiple backends to enable coordinated scheduling. Hermes also leverages the PDGraph model to select optimal backends and prewarm the cold backends. Experimental results on popular workloads show that Hermes can remarkably improve the serving efficiency of LLM applications.]

References

- [1] Alphageometry: An olympiad-level ai system for geometry. <https://deepmind.google/discover/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/>, 2024. [Online; accessed 19-Oct-2024].
- [2] OpenAI Assistants API. <https://platform.openai.com/docs/assistants/overview>, 2024. [Online; accessed 19-July-2024].
- [3] The origin code of code checking. <https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/factool/code>, 2024. [Online; accessed 19-Oct-2024].
- [4] The origin code of code generation. <https://github.com/microsoft/autogen/tree/0560bdd645dfbc579a71f2f0fea98ea83dd3bb3f?tab=readme-ov-file#quickstart>, 2024. [Online; accessed 19-Oct-2024].
- [5] The extra_body parameter supported by openai api. <https://github.com/openai/openai-python/blob/af8e0ad8b7988e72e1d02478022d80eff4f55b91/src/openai/resources/models.py#L51>, 2024. [Online; accessed 19-Oct-2024].
- [6] The extra_body parameter supported by vllm. https://docs.vllm.ai/en/v0.6.0/serving/openai_compatible_server.html#extra-parameters, 2024. [Online; accessed 19-Oct-2024].
- [7] The Shift from Models to Compound AI Systems. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, 2024. [Online; accessed 21-September-2024].
- [8] vLLM: Easy, fast, and cheap LLM serving for everyone. <https://docs.vllm.ai/en/stable/>, 2024. [Online; accessed 19-July-2024].
- [9] OpenAI Function Calling. <https://platform.openai.com/docs/guides/function-calling>, 2025. [Online; accessed 10-Jan-2025].
- [10] Samuli Aalto, Urtzi Ayesta, and Rhonda Righter. On the gittins index in the m/g/1 queue. *Queueing Systems*, 63:437–458, 2009.
- [11] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. Infercept: Efficient intercept support for augmented large language model inference. In *Forty-first International Conference on Machine Learning*, 2024.
- [12] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *USENIX OSDI*, 2024.
- [13] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. {CherryPick}: Adaptively unearthing the best cloud configurations for big data analytics. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 469–482, 2017.
- [14] Matthew Andrews. Probabilistic end-to-end delay bounds for earliest deadline first scheduling. In *IEEE INFOCOM*, 2000.
- [15] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.
- [16] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [17] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- [18] Lingjiao Chen, Matei Zaharia, and James Zou. Frugal-gpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

- [20] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- [21] I Chern, Steffi Chern, Shiqi Chen, Weizhe Yuan, Kehua Feng, Chunting Zhou, Junxian He, Graham Neubig, Pengfei Liu, et al. Factool: Factuality detection in generative ai—a tool augmented framework for multi-task and multi-domain scenarios. *arXiv preprint arXiv:2307.13528*, 2023.
- [22] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- [23] Robert I Davis, Ken W Tindell, and Alan Burns. Scheduling slack time in fixed priority pre-emptive systems. In *1993 Proceedings Real-Time Systems Symposium*, pages 222–231. IEEE, 1993.
- [24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [25] Luciano Floridi and Massimo Chiriatti. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:681–694, 2020.
- [26] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [27] Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. Efficiently serving llm reasoning programs with certaintdex. *arXiv preprint arXiv:2412.20993*, 2024.
- [28] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient llm scheduling by learning to rank. *arXiv preprint arXiv:2408.15792*, 2024.
- [29] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [30] John Gittins, Kevin Glazebrook, and Richard Weber. *Multi-armed bandit allocation indices*. John Wiley & Sons, 2011.
- [31] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. Elasticflow: An elastic serverless training platform for distributed deep learning. In *ACM ASPLOS*, 2023.
- [32] Muhammad Usman Hadi, Rizwan Qureshi, Abbas Shah, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, Seyedali Mirjalili, et al. A survey on large language models: Applications, challenges, limitations, and practical usage. *Authorea Preprints*, 2023.
- [33] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Yuhan Dong, Yu Wang, et al. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. *Proceedings of Machine Learning and Systems*, 6:148–161, 2024.
- [34] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [35] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [36] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [37] Qinghao Hu, Meng Zhang, Peng Sun, Yonggang Wen, and Tianwei Zhang. Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 457–472, 2023.
- [38] Redwan Ibne Seraj Khan, Kunal Jain, Haiying Shen, Ankur Mallick, Anjaly Parayil, Anoop Kulkarni, Steve Kofsky, Pankhuri Choudhary, Renée St Amant, Rujia Wang, et al. Ensuring fair llm serving amid diverse applications. *arXiv e-prints*, pages arXiv–2411, 2024.
- [39] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R Ganger. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *ACM SOSP*, 2023.

- [40] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [41] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. Towards mitigating llm hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1827–1843, 2023.
- [42] Hanlei Jin, Yang Zhang, Dan Meng, Jun Wang, and Jinghua Tan. A comprehensive survey on process-oriented automatic text summarization with exploration of llm-based methods. *arXiv preprint arXiv:2403.02901*, 2024.
- [43] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. s3: Increasing gpu utilization during generative inference for higher throughput. *Advances in Neural Information Processing Systems*, 36:18015–18027, 2023.
- [44] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.
- [45] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with page-dattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [46] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [47] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. Caraserve: Cpu-assisted and rank-aware lora serving for generative llm inference. *arXiv preprint arXiv:2401.11240*, 2024.
- [48] Bin Lin, Tao Peng, Chen Zhang, Minmin Sun, Lanbo Li, Hanyu Zhao, Wencong Xiao, Qi Xu, Xiafei Qiu, Shen Li, et al. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669*, 2024.
- [49] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable. In *USENIX OSDI*, 2024.
- [50] Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services. *arXiv preprint arXiv:2404.16283*, 2024.
- [51] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 1(2):4, 2023.
- [52] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *USENIX OSDI*, 2020.
- [53] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. Queue management for slo-oriented large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*, SoCC '24, page 18–35, New York, NY, USA, 2024. Association for Computing Machinery.
- [54] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*, pages 1–14, 2018.
- [55] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021.
- [56] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [57] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Efficient interactive llm serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509*, 2024.
- [58] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

- [59] Linus Schrage. A proof of the optimality of the shortest remaining processing time discipline. *Operations Research*, 16(3):687–690, 1968.
- [60] Ziv Scully and Mor Harchol-Balter. The gittins policy in the m/g/1 queue. In *2021 19th International Symposium on Modeling and Optimization in Mobile, Ad hoc, and Wireless Networks (WiOpt)*, pages 1–8. IEEE, 2021.
- [61] Philip Sedgwick. Pearson’s correlation coefficient. *Bmj*, 345, 2012.
- [62] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. Don’t stop me now: Embedding based scheduling for llms. *arXiv preprint arXiv:2410.01035*, 2024.
- [63] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2024.
- [64] Zhuocheng Shen. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*, 2024.
- [65] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*, 2023.
- [66] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 965–988, 2024.
- [67] Ramesh Sitaraman. The power of two random choices: A survey of techniques and results. 2001.
- [68] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.
- [69] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *USENIX OSDI*, 2024.
- [70] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Teola: Towards end-to-end optimization of llm-based applications. *arXiv preprint arXiv:2407.00326*, 2024.
- [71] Oguzhan Topsakal and Tahir Cetin Akinci. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International Conference on Applied Engineering and Natural Sciences*, 2023.
- [72] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [73] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654, 2024.
- [74] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- [75] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [76] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *USENIX OSDI*, 2022.
- [77] Youpeng Zhao and Jun Wang. Alise: Accelerating large language model serving with speculative scheduling. *arXiv preprint arXiv:2410.23537*, 2024.
- [78] Peng Zheng, Wendi Feng, Arvind Narayanan, and Zhi-Li Zhang. Nfv performance profiling on multi-core servers. In *2020 IFIP Networking Conference (Networking)*, pages 91–99. IEEE, 2020.
- [79] Pengfei Zheng, Rui Pan, Tarannum Khan, Shivaram Venkataraman, and Aditya Akella. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In *USENIX NSDI*, 2023.
- [80] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An llm-empowered llm inference pipeline. *Advances in Neural Information Processing Systems*, 36, 2024.
- [81] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Dist-Serve: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *USENIX OSDI*, 2024.